

MODERN VIEW OF LINEAR LEAST SQUARES

DANIEL TOPA

ABSTRACT. Present least squares in terms of vectors, new presentation.

CONTENTS

1. The Method of Least Squares	2
1.1. Paeen to Least Squares	2
1.2. Solutions for Linear Systems	3
1.3. $\ \mathbf{A}x - b\ = 0$	4
1.4. The General Solution for Linear Least Squares	6
1.5. Experimental Apparatus	7
1.6. Experimental Measurements	7
2. Least Squares Solution	7
2.1. The General Solution for Linear Least Squares	8
2.2. Numeric Results	9
3. Visualizations	10
3.1. Solution Space	10
3.2. Whisker Plot	10
3.3. Accuracy and Precision	10
4. Exploration	11
4.1. Resolution of Measurement Space	11
4.2. Neighborhood of the Solution	11
4.3. Affine Transform	12
5. Vector Languages	12
5.1. Fortran	12

Date: May 19, 2024.

1991 Mathematics Subject Classification. 26A51, 65F05, 65G99, 93E24.

Key words and phrases. least squares, null spaces.

5.2. Mathematica	14
5.3. Julia	14
5.4. Octave	15
5.5. Python	16
References	16

1. THE METHOD OF LEAST SQUARES

1.1. Paean to Least Squares. One may assert the method of least squares as the greatest endowment of applied mathematics. The toolkit spans the spectrum of mathematical, scientific, and engineering disciplines. It is universal, it is versatile, it is fundamental. It unites different cultures and different fields as a tool of extraordinary power and value. Certainly, intelligent life of other planets will share our valuation of the toolkit as well.

We may view least squares as the collection of tools that regiment the generalization of the concept of equality as well as the concept of matrix inverse. Consider the archetypal matrix–vector equation

$$\mathbf{A}x = b$$

When the data vector b is in the column space of the design matrix $\mathbf{A}$, we may find an exact solution for the vector x . The method of least squares is used when there is no exact solution. Instead, we ask “what is the best solution?” and in doing so, generalize the equality to mean “as close as possible”. Least squares codifies these subjective assessments.

When we apply the powerful singular value decomposition to the least squares problem, we naturally create the Moore–Penrose pseudoinverse matrix in an exercise of stunning ease and breathtaking utility. Such a pseudoinverse generalizes the matrix inverse to cover situations where we have generalized the equality, that is, to cases where the data vector is not in the column space of $\mathbf{A}$.

The well-known Fredholm Alternative theorem is a simple statement about the need to use least squares. It tells us that when the data vector is in the column space of the matrix, we may use elementary tools to find an exact solution, otherwise, we must use the method of least squares. We will learn that the Fredholm Alternative says we must turn to least squares when there is a null space component to the data vector.

We see signs alluding to the fact that understanding the method of least squares demands an understanding of the invariant subspaces induced by the design matrix. Doing so leads immediately to Fundamental Theorem of Linear Algebra. In service of this objective, care has been taken to color vectors to immediately identify their pedigree as **range space** or **null space** objects. In this fashion, we vitalize the fundamental subspaces. Mixed vectors retain the default black coloring.

My favorite property is existence: there is always a least squares solution. This can lead to delicious conundrums such as finding the least squares solution when the inputs are two parallel lines. What is the best line through a single point? Besides being fun examples to work, they cleanly demonstrate basic properties of least squares.

In short order, we understand that the most powerful theoretical tools from linear algebra, the Fundamental Theorem of Linear Algebra, the Fredholm Alternative, and the singular value decomposition form the bedrock upon which the method of least squares rests.

We examine the many different guises of the least squares problem; we explore many different tools. We unify the presentation of results for so many disparate cases to show the shared mantle of least squares. The folk lore of least squares is prodigious and, at times, misleading. Our goal is to demonstrate the elegance and utility of formal employment.

1.2. Solutions for Linear Systems. hermeneutic Our story begins with the archetypal matrix–vector equation

$$(1) \quad \mathbf{A}x = b$$

The matrix $\mathbf{A}$ has m rows, n columns, and has rank ρ ; the vector b encodes m measurements. The solution vector x represents the n free parameters in the modal. In mathematical shorthand,

$$(2) \quad \begin{array}{ccc} \mathbf{A} \in \mathbb{C}_{\rho}^{m \times n}, & b \in \mathbb{C}^m, & x \in \mathbb{C}^n \\ \text{mesh} & \text{measurements} & \text{solution} \end{array}$$

with $\mathbb{C}$ representing the field of complex numbers. The matrix encodes information about the mesh, that is, the location of sample points. The data vector b contains the measurements, and the solution vector x provides the free parameters in the modal.

We restrict our concern to the forward problem where the system matrix $\mathbf{A}$ and the data vector b are given, and the task is to find the solution vector x .

TABLE 1. Forward and inverse problems.

classification	given	find
forward problem	$\mathbf{A}, x$	b
inverse problem	$\mathbf{A}, b$	x

Linear algebra solutions form a strata of increasing complexity. The simplest stratification criterion comes from r , the residual error vector:

$$r(x) = \mathbf{A}x - b$$

If each component of this residual error vector is zero, the solution vector x represents an *exact* solution. If any component of r is not zero, the solution is *approximate*. We may summarize the distinction in terms of norms:

TABLE 2. Solution types and solution methods categorized by total error.

total error	solution type	solution method
$\ r\ = 0$	exact	direct and least squares
$\ r\ > 0$	approximate	least squares exclusively



FIGURE 1. The methods of least squares rely upon the orthogonal decomposition of the fundamental subspace. Vectors in range spaces ($\mathcal{R}(\mathbf{A})$ or $\mathcal{R}(\mathbf{A}^*)$) are colored in blue. Vectors in null spaces ($\mathcal{N}(\mathbf{A}^*)$ or $\mathcal{N}(\mathbf{A})$) are colored in red. Amalgam vectors with components in both spaces are the traditional black.

1.3. $\|\mathbf{A}x - b\| = 0$. The matrix–vector multiplication in $\boxed{\mathbf{I}}$ represents a process whereby a matrix operates on an n –vector and returns an m –vector. We can think of the matrix as a map from vectors of dimension n to vectors of dimension m :

$$\mathbf{A}: \mathbb{C}^n \mapsto \mathbb{C}^m.$$

If the vector b can be expressed a combination of the columns of the matrix $\mathbf{A}$ then the linear system is *consistent* and there is an exact solution:

$$\mathbf{A}x = b \implies x_1 a_1 + \cdots + x_n a_n = b$$

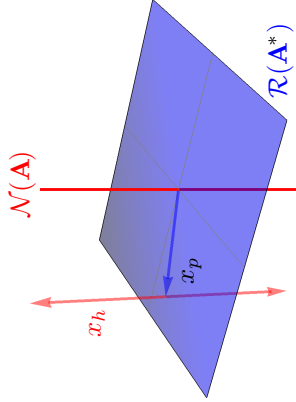
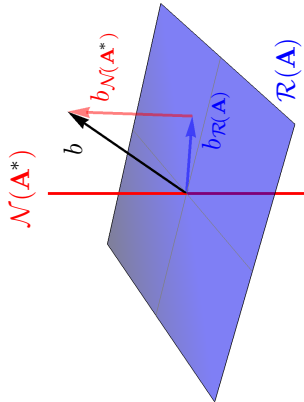
and the residual error vanishes:

$$r = \mathbf{A}x - b = \mathbf{0}.$$

The zero vector $\mathbf{0}$ is a vector of m zeros. The total error, the norm of the r vector, is 0:

$$\|r\| = \|\mathbf{A}x - b\| = 0.$$

TABLE 3. The general Least Squares Problem in pictures. The data vector, b (right), extends into the range and null space elements of the measurement space. The Least Squares Solution identifies $b_{\mathcal{R}(\mathbf{A})}$, the range space component of the data vector. The particular solution, x_p (dot, left), the element in the range of the solution, which $\mathbf{A}$ maps to $b_{\mathcal{R}(\mathbf{A})}$. That is, the solution is the vector which exactly solves $\mathbf{A}x_p = b_{\mathcal{R}(\mathbf{A})}$. The homogenous solution, x_h , is the affine space $\mathcal{N}(\mathbf{A})$, anchored at x_p (dashed red line)

Solution space	Mapping	Data space
Row space n-vector  $\mathbb{C}^n = \mathcal{R}(\mathbf{A}^*) \oplus \mathcal{N}(\mathbf{A})$	$\mathbf{A} x_p \mapsto b_{\mathcal{R}(\mathbf{A})}$	Column space m-vector  $\mathbb{C}^m = \mathcal{R}(\mathbf{A}) \oplus \mathcal{N}(\mathbf{A}^*)$

1.4. The General Solution for Linear Least Squares. Given a matrix and its pseudoinverse, $\mathbf{A}$ and $\mathbf{A}^\dagger$, Laub provides the general least squares solution in both his books [1, 2], coincidentally with the same number eq. 8.1] for (??) as

$$(3) \quad x_{LS} = x_p + (\mathbf{I}_n - \mathbf{A}^\dagger \mathbf{A}) z, \quad z \in \mathbb{R}^n$$

Partial Differential Equation

$$(4) \quad T_t(x, t) = \alpha \Delta T(x, t)$$

Dirichlet Boundary Conditions

$$(5) \quad \begin{aligned} T(x=0, t) &= 0, \\ T(x=100, t) &= 100, \end{aligned}$$

$$(6) \quad \lim_{t \rightarrow \infty} T_t(x, t) = 0$$

$$(7) \quad \begin{aligned} T_t(x, t) &= \alpha \Delta T(x, t) \\ T_t &= 0, \end{aligned}$$

...is given by Laplace's Equation

$$(8) \quad 0 = \alpha \Delta T(x, t)$$

$$(9) \quad 0 = \alpha \Delta T(x, t), \quad 0 \leq x \leq 10$$

$$(10) \quad T(x) = 0 + 100x$$

Let's generalize the solution

$$(11) \quad \begin{aligned} T(x) &= 0 + 100x \\ &\Downarrow \\ T(x) &= a_0 + a_1 x \end{aligned}$$

Inputs:

- (1) Model function, $T(x) = a_0 + a_1 x$
- (2) Sequence of measurements, $\{x_k, T_k\}_{k=1}^m$

Outputs: Model parameters

- (1) $a_0 \pm \sigma_0$, intercept with error
- (2) $a_1 \pm \sigma_1$, slope with error

TABLE 4. Bevington's input data and results.

k	x_k	Measurement	Prediction	
		(cm) (°C) T_k	(°C) (°C) $T(x_k)$ r_k	
1	1	15.6	14.2	-1.4
2	2	17.5	23.6	6.1
3	3	36.6	33.0	-3.6
4	4	43.8	42.4	-1.4
5	5	58.2	51.9	-6.3
6	6	61.6	61.3	-0.4
7	7	64.2	70.7	6.5
8	8	70.4	80.1	9.7
9	9	98.8	89.5	-9.3

1.5. Experimental Apparatus. Temperature measurements are made on an equipartition of a thermal conductor connecting two heat baths at constant and distinct temperatures. Such an arrangement is depicted in figure ??, with the upward facing arrowheads (gray) marking measurement positions.

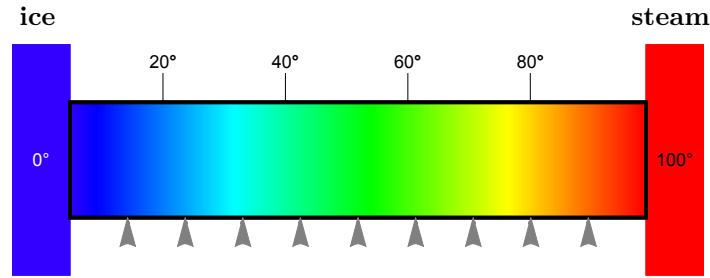


FIGURE 2. The experimental apparatus is designed to measure a the temperate gradient between two well-characterized constant temperature heat baths.

1.6. Experimental Measurements.

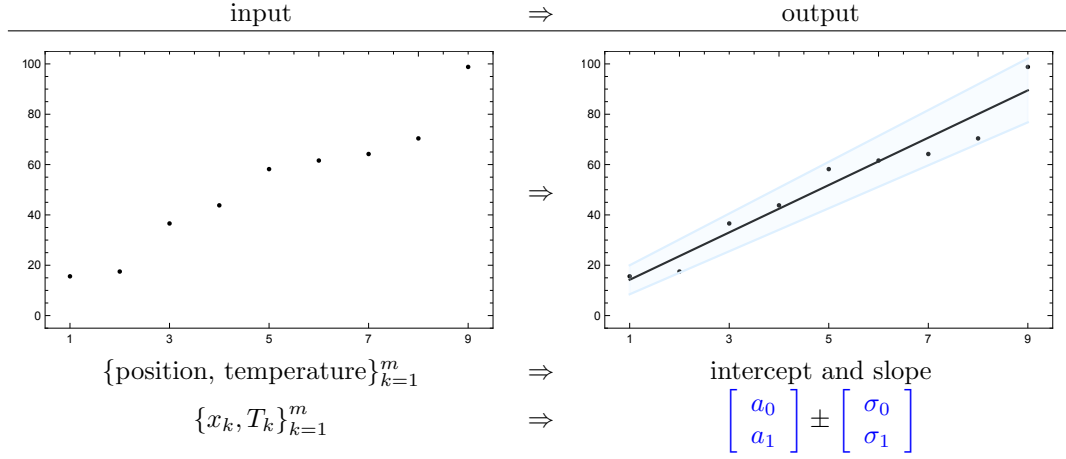
2. LEAST SQUARES SOLUTION

The [Least Squares Solution](#) is defined as

$$x_{LS} = \arg \min_{x \in \mathbb{C}^n} \|\mathbf{A}x - b\|_2$$

The [Least Squares Solution](#) is expressed as

$$\begin{aligned}
 x_{LS} &= \mathbf{A}^\dagger b &+& (\mathbf{I}_n - \mathbf{A}^\dagger \mathbf{A})y \\
 &= \mathbf{A}^\dagger b &+& \mathbf{P}_{\mathcal{N}(\mathbf{A})}y \\
 &= x_p &+& x_h \\
 &= x_{\mathcal{R}(\mathbf{A}^*)} &+& x_{\mathcal{N}(\mathbf{A})}
 \end{aligned}
 \tag{12}$$



Least Squares Residual Error Vector

$$(13) \quad \mathbf{r}(x) = \mathbf{A}x - b$$

Least Squares Merit Function

$$(14) \quad M(x) = \langle \mathbf{r}(x), \mathbf{r}(x) \rangle = \langle \mathbf{A}x - b, \mathbf{A}x - b \rangle$$

Least Total Squared Error

$$(15) \quad t^2 = \langle \mathbf{r}(x_{LS}), \mathbf{r}(x_{LS}) \rangle$$

Least Squares Uncertainty (Standard Deviation)

$$(16) \quad \sigma_k^2 = \frac{t^2}{m - n} (\mathbf{A}^* \mathbf{A})_{k,k}^{-1}, \quad k = 1:n$$

$$(17) \quad \begin{aligned} \langle x, y \rangle &= x^* y = x \cdot y = \sum \bar{x}_k y_k \\ &= \int_{\Omega} \bar{x} y d\Omega \end{aligned}$$

$$(18) \quad \begin{aligned} \mathbf{A}^* \mathbf{A} \quad a &= \mathbf{A}^* T \\ \begin{bmatrix} \langle \mathbf{1}, \mathbf{1} \rangle & \langle \mathbf{1}, x \rangle \\ \langle x, \mathbf{1} \rangle & \langle x, x \rangle \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} &= \begin{bmatrix} \langle \mathbf{1}, T \rangle \\ \langle x, T \rangle \end{bmatrix} \\ \begin{bmatrix} 9 & 45 \\ 45 & 285 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} &= \begin{bmatrix} 466.7 \\ 2898 \end{bmatrix} \end{aligned}$$

2.1. The General Solution for Linear Least Squares. Given a matrix and its pseudoinverse, $\mathbf{A}$ and $\mathbf{A}^\dagger$, Laub provides the general least squares solution in both his books [1] [2] coincidentally with the same number eq. 8.1] for (??) as

$$(19) \quad x_{LS} = x_p + (\mathbf{I}_n - \mathbf{A}^\dagger \mathbf{A}) z, \quad z \in \mathbb{R}^n$$

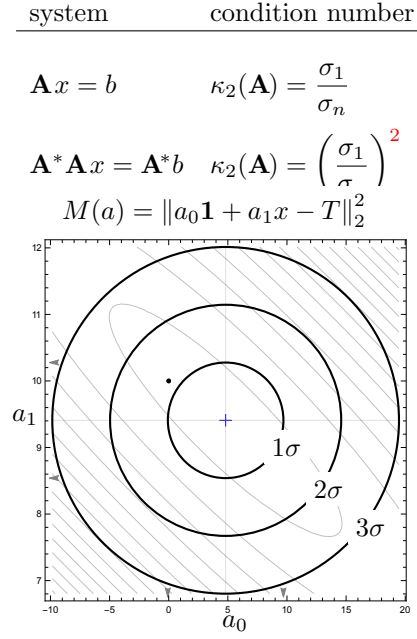


FIGURE 3. example caption

2.2. Numeric Results.

$$\begin{aligned}
 \sigma_0 &= 4.886206312183354 && \text{rounded to 4.9,} \\
 a_0 &= 4.813888888888889 && \text{rounded to 4.8;} \\
 \sigma_1 &= 0.8683016476563611 && \text{rounded to 0.87,} \\
 a_1 &= 9.408333333333333 && \text{rounded to 9.41.}
 \end{aligned}
 \tag{20}$$

Either

$$a_p = \begin{bmatrix} a_0 \\ a_1 \end{bmatrix}_p \pm \begin{bmatrix} \sigma_0 \\ \sigma_1 \end{bmatrix} = \begin{bmatrix} 4.8 \\ 9.41 \end{bmatrix} \pm \begin{bmatrix} 4.9 \\ 0.87 \end{bmatrix}
 \tag{21}$$

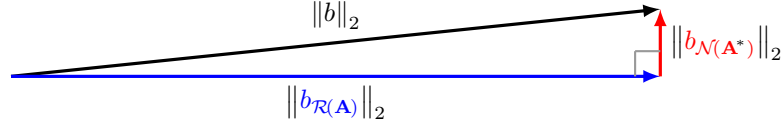
Or

$$a_p = \begin{bmatrix} a_0 \pm \sigma_0 \\ a_1 \pm \sigma_1 \end{bmatrix}_p = \begin{bmatrix} 4.8 \pm 4.9 \\ 9.41 \pm 0.87 \end{bmatrix}
 \tag{22}$$

TABLE 5. default

$$\begin{aligned}
 \langle \mathbf{1}, \mathbf{1} \rangle &= 9 \\
 \langle \mathbf{1}, x \rangle &= 45 & \langle x, x \rangle &= 45 & \langle x, T \rangle &= 45 \\
 \langle \mathbf{1}, T \rangle &= 466.7 & \langle T, T \rangle &= 29828.65
 \end{aligned}$$

$$\mathbb{S} = \frac{\|b_{\mathcal{R}(\mathbf{A})}\|_2}{\|b_{\mathcal{N}(\mathbf{A}^*)}\|_2} = \frac{171.791}{17.795} \approx 9.65.
 \tag{23}$$



3. VISUALIZATIONS

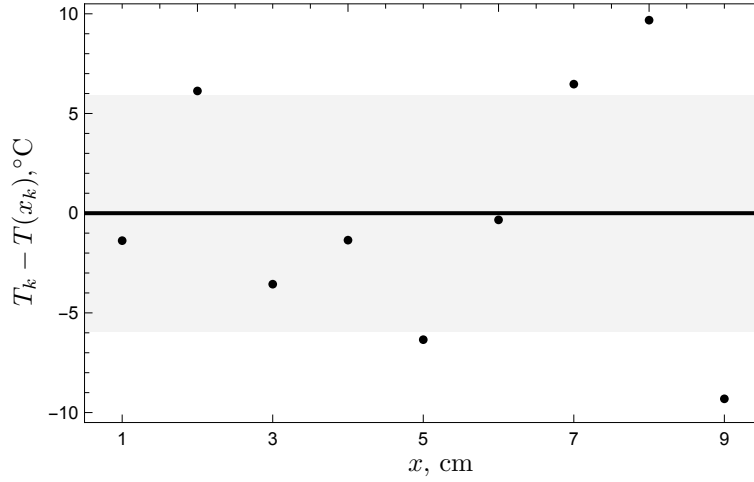
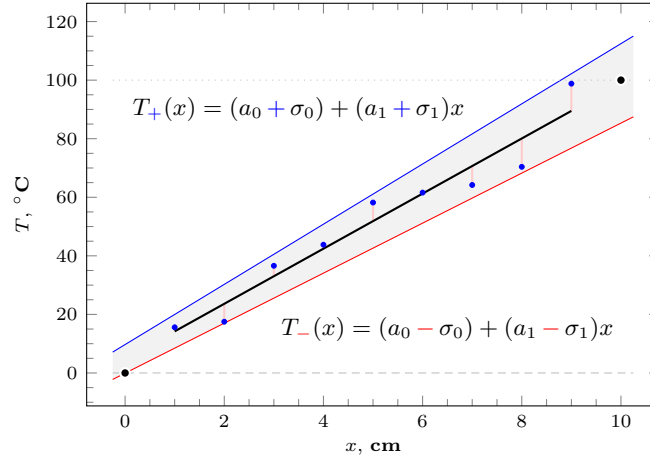


FIGURE 4. default

3.1. Solution Space.

3.2. Whisker Plot.

3.3. Accuracy and Precision.

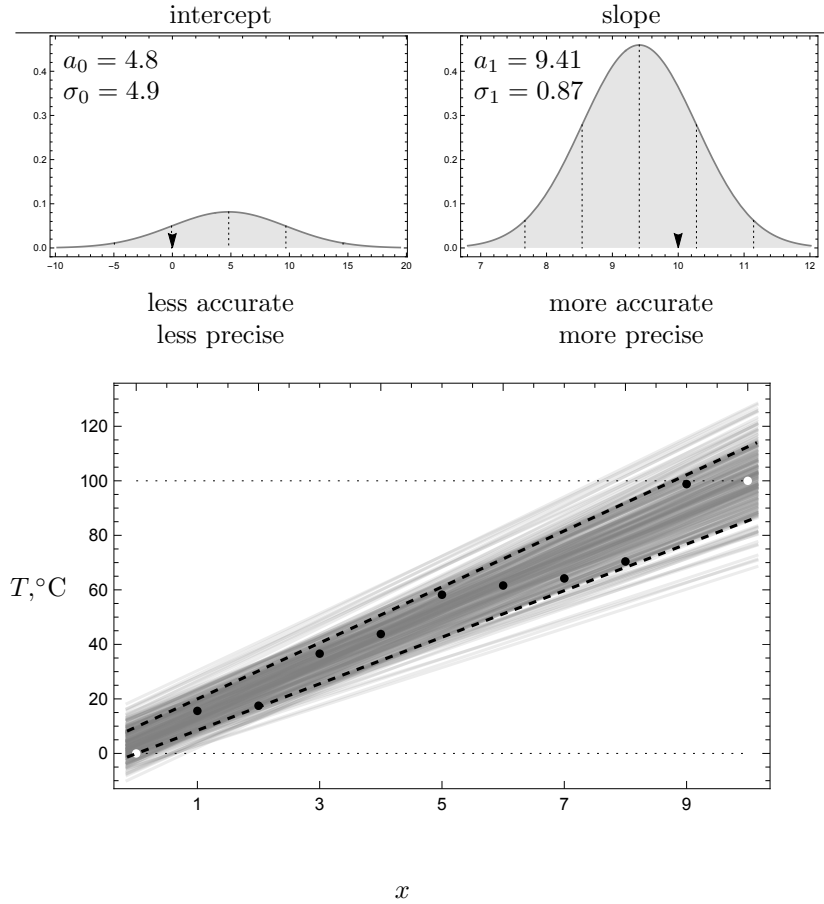


FIGURE 5. default

4. EXPLORATION

4.1. Resolution of Measurement Space.

$$\begin{aligned}
 \mathbf{A} &= \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \\ 1 & 6 \\ 1 & 7 \\ 1 & 8 \\ 1 & 9 \end{bmatrix} & a_p &= \begin{bmatrix} 4.8 \\ 9.41 \end{bmatrix} &= & \frac{1}{360} \begin{bmatrix} 5120 \\ 8507 \\ 11894 \\ 15281 \\ 18668 \\ 22055 \\ 25442 \\ 28829 \\ 32216 \end{bmatrix} &= & \frac{1}{360} \begin{bmatrix} 5616 \\ 6300 \\ 13176 \\ 15768 \\ 20952 \\ 22176 \\ 23112 \\ 25344 \\ 35568 \end{bmatrix} &= & \frac{1}{360} \begin{bmatrix} 496 \\ -2207 \\ 1282 \\ 487 \\ 2284 \\ 121 \\ -2330 \\ -3485 \\ 3352 \end{bmatrix} \\
 & & & & & 171.791^2 &= & 172.710^2 &= & 17.795^2
 \end{aligned}
 \tag{24}$$

4.2. Neighborhood of the Solution. How does the merit function change as the least squares solution is perturbed?

$$M(a_p + \delta) - M(a_p) \geq 0. \tag{25}$$

$$\begin{aligned}
 a_0 &\rightarrow a_0 + \delta, \\
 a_1 &\rightarrow a_1 + \delta.
 \end{aligned}
 \tag{26}$$

	$M(a_p + \delta)$ perturbed	—	$M(a_p)$ unperturbed	=	ζ difference
1	$(a_0 + \delta)^2 \langle \mathbf{1}, \mathbf{1} \rangle$	—	$a_0^2 \langle \mathbf{1}, \mathbf{1} \rangle$	=	$(2a_0\delta + \delta^2) \langle \mathbf{1}, \mathbf{1} \rangle$
2	$(a_1 + \delta)^2 \langle x, x \rangle$	—	$a_1^2 \langle x, x \rangle$	=	$(2a_1\delta + \delta^2) \langle x, x \rangle$
3	$\langle T, T \rangle$	—	$\langle T, T \rangle$	=	0
4	$2(a_0 + \delta)(a_1 + \delta) \langle \mathbf{1}, x \rangle$	—	$2a_0a_1 \langle \mathbf{1}, x \rangle$	=	$2((a_0 + a_1)\delta + \delta^2) \langle \mathbf{1}, x \rangle$
5	$-2(a_0 + \delta) \langle T, T \rangle$	—	$-2a_0 \langle T, T \rangle$	=	$-2\delta \langle T, T \rangle$
6	$-2(a_1 + \delta) \langle x, T \rangle$	—	$-2a_1 \langle x, T \rangle$	=	$-2\delta \langle x, T \rangle$
Σ	$t^2 + \zeta$	—	t^2	=	ζ

Tedious algebra follows; **no magic** here.

$$\begin{aligned}
 (27) \quad M(a) &= \sum_{k=1}^m (T(x_k) - T_k)^2 = \sum_{k=1}^m (a_0 + a_1 x_k - T_k)^2 \\
 &= (a_0 \mathbf{1} + a_1 x - T)^T (a_0 \mathbf{1} + a_1 x - T) \\
 &= \langle a_0 \mathbf{1} + a_1 x - T, a_0 \mathbf{1} + a_1 x - T \rangle
 \end{aligned}$$

$$\begin{aligned}
 (28) \quad M(a) &= \langle r(a), r(a) \rangle = \langle a_0 \mathbf{1} + a_1 x - T, a_0 \mathbf{1} + a_1 x - T \rangle \\
 &= a_0^2 \underset{1}{\langle \mathbf{1}, \mathbf{1} \rangle} + a_1^2 \underset{2}{\langle x, x \rangle} + \underset{3}{\langle T, T \rangle} + 2a_0a_1 \underset{4}{\langle \mathbf{1}, x \rangle} \\
 &\quad - 2a_0 \underset{5}{\langle \mathbf{1}, T \rangle} - 2a_1 \underset{6}{\langle x, T \rangle}
 \end{aligned}$$

$$\begin{aligned}
 (29) \quad \zeta &= \delta^2 (\langle x, x \rangle + 2\langle \mathbf{1}, x \rangle + \langle \mathbf{1}, \mathbf{1} \rangle) \\
 &= \delta^2 \langle x + \mathbf{1}, x + \mathbf{1} \rangle \geq 0
 \end{aligned}$$

- Compute a_0, a_1
- Compute t^2
- Input δ
- Compute $t^2 + \zeta$
- Demonstrate $\zeta = \delta^2 \langle x + \mathbf{1}, x + \mathbf{1} \rangle$

Next, take a data sequence with an **ideal solution** ($t^2 = 0$) and **perturb a single point**. Perturbation:

$$(30) \quad T_k \rightarrow T_k + \delta$$

Implies

$$\begin{aligned}
 (31) \quad \langle x, T \rangle &\rightarrow \langle x, T \rangle + x_k \delta, \\
 \langle \mathbf{1}, T \rangle &\rightarrow \langle \mathbf{1}, T \rangle + \delta.
 \end{aligned}$$

With modest effort equation ?? becomes

$$\begin{aligned}
 (32) \quad a_0 &\rightarrow a_0 + \frac{\delta}{\Delta} (\langle x, x \rangle - \langle \mathbf{1}, x \rangle x_k), \\
 a_1 &\rightarrow a_1 + \frac{\delta}{\Delta} (-\langle \mathbf{1}, x \rangle + \langle \mathbf{1}, \mathbf{1} \rangle x_k).
 \end{aligned}$$

4.3. Affine Transform.

5. VECTOR LANGUAGES

5.1. **Fortran**. Fortran was the first modern language for numerical computation and over many decades has evolved into a thoroughly capable and modern tool. The vector tools are fast and easy to use. The speed of execution of compiled code is the standard used to gauge performance of other languages.

In the text, we mentioned that a true vector toolkit does not need to specify the dimensions of the components and that this alleviates many software bugs. We then describe Fortran as a true vector toolkit. So the engaged read may wonder why we specify the dimensions of our matrices and vectors in the declarations. This was a choice from

writing the simplest example. In code written for computation, the computation would be in subroutines which would operate on deferred shapes. For example `x(1:m)` would become `x(*)`.

```

1  ! 3456789 123456789 223456789 323456789 423456789 523456789 623456789 723456789 823456789 923456789 023456789 123456789 223
2  program least_squares
3
4      use, intrinsic :: iso_fortran_env, only : REAL64
5
6      implicit none
7
8      integer, parameter :: rp = selected_real_kind ( REAL64 ), m = 9, n = 2
9
10     ! rank one
11     real ( kind = rp ), parameter :: one ( 1 : m ) = &
12         [ 1.0_rp, 1.0_rp, 1.0_rp, 1.0_rp, 1.0_rp, 1.0_rp, 1.0_rp, 1.0_rp, 1.0_rp ]
13     real ( kind = rp ), parameter :: x ( 1 : m ) = &
14         [ 1.0_rp, 2.0_rp, 3.0_rp, 4.0_rp, 5.0_rp, 6.0_rp, 7.0_rp, 8.0_rp, 9.0_rp ]
15     real ( kind = rp ), parameter :: T ( 1 : m ) = &
16         [ 15.6_rp, 17.5_rp, 36.6_rp, 43.8_rp, 58.2_rp, 61.6_rp, 64.2_rp, 70.4_rp, &
17             98.8_rp ]
18
19     ! scalars
20     real ( kind = rp ) :: a = 0.0_rp, b = 0.0_rp, c = 0.0_rp, d = 0.0_rp, e = 0.0_rp, det = 0.0_rp, a0 = 0.0_rp, a1 = 0.0_rp
21
22     ! inner products
23     a = dot_product ( one, one )
24     b = dot_product ( one, x )
25     c = dot_product ( x, x )
26     d = dot_product ( one, T )
27     e = dot_product ( x, T )
28     ! determinant of Gram matrix
29     det = a * c - b**2
30
31     ! equations 10.26
32     a0 = ( d * c - b * e ) / det
33     a1 = ( a * e - b * d ) / det
34
35     write ( *, * ) "particular least squares solution"
36     write ( *, * ) "intercept a0 = ", a0
37     write ( *, * ) "slope a1 = ", a1
38
39     stop 'Successful termination for "least_squares.f08".'
40 end program least_squares

```

The code produces the following results:

```

particular least squares solution for ground zero
x0 =    117.97723925949958      +/-    18.698052680295692
y0 =    135.24677447077607      +/-    20.860166494699182

Errors in machine epsilon =    2.2204460492503131E-016
x0 =   -256.00000000000000      +/-   -32.000000000000000

```

$y_0 = -256.00000000000000 \quad +/ - \quad -32.00000000000000$

5.2. Mathematica.

5.3. [Julia](#) Julia is a high-level, dynamic programming language with features and tools well suited for numerical analysis and computational science.

```

1 # define mesh
2 x = [ 1; 20; 50 ]
3 y = [ 66; 15; -3 ]
4 # bearing angles (radians)
5 theta = [ 1.0; 0.74; 0.43 ]
6
7 # data vectors
8 c = cos.( theta )
9 s = sin.( theta )
10
11 # construct linear system
12 A = [ c'; -s' ]'
13 b = x .* c - y .* s
14
15 # load linear algebra package for dot product
16 using LinearAlgebra
17 cc = dot(c, c)
18 cs = dot(c, s)
19 ss = dot(s, s)
20 cb = dot(c, b)
21 sb = dot(s, b)
22
23 det = cc * ss - cs^2
24
25 # normal equations solution
26 Winv = [ ss cs; cs cc] / det
27 atb = [ cb; -sb ]
28 xls = Winv * atb
29
30 # error propagation
31 # residual error vector
32 residual = A * xls - b
33 t2 = dot( residual, residual )
34 # harvest diagonal elements
35 values = diag( Winv )
36 # measurements (rows) and free variables (columns)
37 ( m, n ) = size( A )
38 # compute error elements
39 sigma = sqrt.( t2 / ( m - n ) * values )

```

The code produces the following results:

```
particular least squares solution for ground zero
xls = [117.97723925949958, 135.2467744707761]
+/- [18.698052680295692, 20.860166494699182]
```

```
errors in intercept and slope as machine epsilon sigmas
xls = [-256.0, -128.0] +/- [-32.0, -32.0]
```

5.4. [Octave](#). Partial listing of the Octave code to show primary computations.

```
1 printf( "Finding ground zero with bearing angles\n" )
2
3 # Define mesh
4 x = [ 1; 20; 50 ];
5 y = [ 66; 15; -3 ];
6 # bearing angles (radians)
7 theta = [ 1.0; 0.74; 0.43 ];
8
9 # data vectors
10 c = cos( theta );
11 s = sin( theta );
12
13 # form linear system Ax = b
14 A = [ c, -s ];
15 b = x .* c - y .* s;
16
17 # inner products
18 cc = dot(c, c);
19 cs = dot(c, s);
20 ss = dot(s, s);
21 cb = dot(c, b);
22 sb = dot(s, b);
23
24 det = cc * ss - cs^2;
25
26 # normal equations solution
27 Winv = [ ss cs; cs cc] / det;
28 atb = [ cb; -sb ];
29
30 # least squares solution: xls
31 xls = Winv * atb;
32
33 # residual error vector
34 residual = A * xls - b;
35
36 t2 = dot( residual, residual );
37 values = diag( Winv );
38 sigma = sqrt( t2 / ( m - n ) * values );
```

The code produces the following results:

```

particular least squares solution for ground zero
x0 = 117.977239259499584 +/- 18.698052680295692
y0 = 135.246774470776103 +/- 20.860166494699182

```

```

Errors in machine epsilon = 2.22045e-16
x0 = -256 +/- -32
y0 = -128 +/- -32

```

5.5. [Python](#). Partial listing of the Python code to show primary computations.

```

1 import numpy as np
2
3 # input data
4 x = np.array( [ 1, 20, 50 ] );
5 y = np.array( [ 66, 15, -3 ] );
6 theta = np.array( [ 1.0, 0.74, 0.43 ] );
7
8 # data vectors
9 c = np.cos( theta );
10 s = np.sin( theta );
11
12 # form linear system Ax = b
13 A = np.transpose( [ c, -s ] )
14 b = np.multiply( x, c ) - np.multiply( y, s )
15
16 # inner products
17 cc = np.dot( c, c );
18 cs = np.dot( c, s );
19 ss = np.dot( s, s );
20 cb = np.dot( c, b );
21 sb = np.dot( s, b );
22
23 det = cc * ss - cs**2
24
25 # normal equations solution
26 Winv = [ [ ss, cs ], [ cs, cc ] ] / det;
27 atb = [ cb, -sb ];
28
29 # least squares solution: xls
30 xls = np.dot( Winv, atb );
31 print ( "least squares solution xls =", xls )

```

REFERENCES

- [1] Alan J. Laub. *Computational Matrix Analysis*. SIAM, 2012.
- [2] Alan J. Laub. *Matrix Analysis for Scientists and Engineers*. SIAM, 2005.